SENAI - FATESG
Análise de Desenvolvimento de Software
Turma ADS3 / 2026-1
Arquitetura e Projeto de Software: **Agenda Fatesg**

METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE

Documento de Arquitetura de Software

Agenda Fatesg	
Gestor do Projeto	**Gerentes de Projeto**
Lucio Aguiar de Souza	Guilherme Silva e Jurandir Marques
luciaguiardesouzaadsturma1@gmail.com	jurandirnetosenai@gmail.com gui21488@gmail.com
62992408578	62987514423 | 62978463214

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
11/02/2026	APS00001	Clayton Marques Pereira	Introdução da camada Controller e extração de lógica da UI	001
20/02/2026	APS00002	Lucio Aguiar de Souza Guilherme Silva Costa Jurandir Marques	Extração de Service, Repository, Implementação de Validation, Result Pattern e Documentação via ADRs.	002

Sumário

1. INTRODUÇÃO 3
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 3
2. REPRESENTAÇÃO ARQUITETURAL 4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS 4
4. VISÃO DE CASOS DE USO 5
4.1 Casos de Uso significantes para a arquitetura 6
5. VISÃO LÓGICA 7
5.1 Panorama geral 7
5.2 Camadas e responsabilidades 7
5.2.1 Apresentação (UI) 7
5.2.2 Controllers 7
5.2.3 Serviços (Domain/Service) 7
5.2.4 Persistência (Repositories) 7
5.3 Organização em pacotes / subsistemas 7
5.4 Coesão e acoplamento 7
5.5 Classes mais importantes 7
5.5.1 Form_Listagem / Form_Cadastro 7
5.5.2 IContatoCadastroController e IContatoListaController (interface) 7
5.5.3 ContatoController 7
5.5.4 ContatoService e ContatoValidation 7
5.5.5 ContatoRepository 7
5.6 Realização dos casos de uso 8
5.6.1 UC03 Criar Contato: 8
5.6.2 UC01 Listar Contatos: 8
5.6.3 UC05 Excluir (marcar inativo) 8
5.7 Concorrência / performance 8
5.8 Visão Geral – pacotes e camadas 9
6. VISÃO DE IMPLEMENTAÇÃO 10
6.1 Caso de Uso [UC03 - Criar Novo Contato] 10
6.1.1 Diagrama de Classes 10
6.1.2 Diagrama de Sequência 10
7. VISÃO DE IMPLANTAÇÃO 11
Descrever os modos físicos, as configurações e os artefatos que serão implantados 11
8. DIMENSIONAMENTO E PERFORMANCE 11
8.1 *Volume (estimativas e contexto)* 11
8.2 *Performance (metas e requisitos)* 12
8.3 *Medição e monitoramento (como validar)* 12
8.4 *Estratégias de otimização (curto e médio prazo)* 12
8.5 *Riscos de performance e mitigação* 12
8.6 *Efetividade na Execução* 12
8.7 Valores base para relatórios arquiteturais 12
9. QUALIDADE 13

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema Agenda de contatos, usando diversas visões de arquitetura para **representar** diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema.
O documento irá adotar uma estrutura baseada na visão "4+1" de modelo de arquitetura [KRU41].

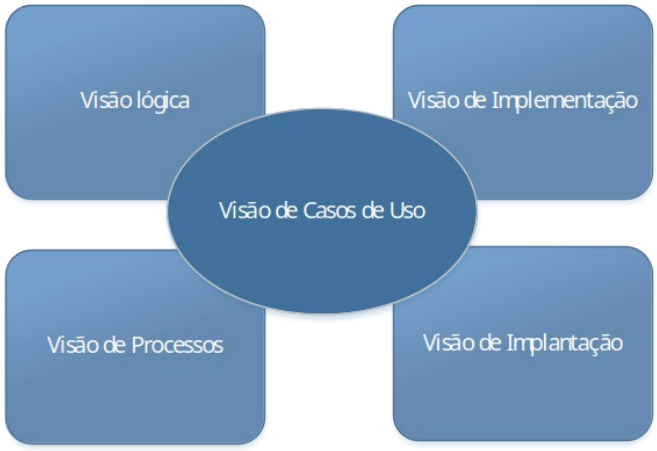

Figura 1 – Arquitetura 4+1

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao **Agenda de contatos.**

1.3 Definições, Acrônimos e Abreviações

QoS – Quality of Service, ou qualidade de serviço. Termo utilizado para descrever um conjunto de qualidades que descrevem as requisitos não-funcionais de um sistema, como performance, disponibilidade e escalabilidade[QOS].

1.4 Referências

[KRU41]: The "4+1" view model of software architecture, Philippe Kruchten, November 1995, http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf

[QOS] https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html

[ADR-0000]: Architecture Decision Record 0000 - Implementação de Exclusão Lógica e Purging.

[ADR-0001]: Architecture Decision Record 0001 - Abstração Controller-First.

[ADR-0002]: Architecture Decision Record 0002 - Extração do Service e Repository.

[ADR-0003]: Architecture Decision Record 0003 - Criação de Validation e Hierarquia de CoreException.

[ADR-0004]: Architecture Decision Record 0004 - Implementação do Unified Response Pattern.

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	Diagrama de Classes
Processo	Integradores	Performance, Escalabilidade, Concorrência	Diagrama de Sequência
Implementação	Programadores	Componentes de Software	Diagrama de Componentes
Implantação	Gerência de Configuração	Nodos físicos	Diagrama de Implantação
Caso de Uso	Todos	Requisitos funcionais	Diagrama de Casos de Uso
Dados	Especialistas em dados Administradores de dados	Persistência de dados	agenda.txt (TXT File)

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descrever os requisitos de software e restrições que tem um impacto significante na arquitetura.

Requisito	Solução
Linguagem	Java 21 (JDK 21). Código compatível com características do Java 21; Builds via Maven (pom.xml). Uso de bibliotecas padrão (Swing, java.nio.file).
Plataforma	Aplicação desktop executada na JVM do cliente (desktop) Distribuível: JAR executável. Observações: requisito mínimo: JVM 21 instalada na máquina do usuário. Não há servidor de aplicação; execução local. Instruções de execução: java -jar Agenda_fatesg.jar (ou execução via IDE). Arquivos de dados armazenados localmente
Segurança	Segurança arquitetural baseada no bloqueio de fluxo de dados incorretos via Classes Interceptadoras de Domínio (ContatoValidation). As validações não habitam e não sujam o Controller, lançando exceptions de regra rigorosas capturadas pelo Response Pattern (ADR-0004 e ADR-0003). Dados são persistidos em formato plano texto localmente, assumindo a premissa de que a segurança física do arquivo agenda.txt é delegada ao Sistema Operacional (File Permissions) na máquina Desktop do usuário.
Persistência	A persistência não ocorre mais no Controller (Plano futuro concluído de acordo com a ADR-0002). O projeto foi desmembrado num ContatoRepository que encapsula a leitura e escrita do agenda.txt garantindo o Single Responsibility Principle (Princípio da Responsabilidade Única). As validações e tratativas sintáticas ocorrem no ContatoValidation que retorna CoreExceptions personalizadas limitando o tráfego incorreto para a base em texto.
Internacionalização (i18n)	Não há necessidade imediata de i18n neste estágio (interface e mensagens em Português-BR).

Tabela 2 – Requisitos e restrições

4. VISÃO DE CASOS DE USO

Agenda Fatesg (Arquitetura em Camadas com Repository Isolado)

Resumo: Atores, casos de uso principais e vínculo guiado pelo MVC Response e Injeção de Dependências.

- **Atores**
    - Usuário Final (Utilizador da Agenda) — Interage passivamente com a aplicação GUI em Swing.
    - Sistema Operacional (I/O Local) — Responde ativamente pelas rotinas de java.nio no FileSystem.

Lista de casos de uso do sistema:

- UC01 — Abrir aplicação / Exibir listagem
- UC02 — Buscar/Filtrar contatos por nome
- UC03 — Criar novo contato
- UC04 — Editar contato existente
- UC05 — Excluir / Marcar contato inativo
- UC06 — Reativar contato inativo (opcional)
- UC07 — Filtrar localmente (UI): já descrito no UC02 — implementado via TableRowSorter no cliente.
- UC08 — Navegar entre telas (Listagem ↔ Cadastro): transição UI (ListView cria FormView e dispose da ListView).

Requisitos Funcionais (Rfs):

- RF1 - O sistema deve permitir a inclusão e edição de novos contatos.
- RF2 - O sistema deve permitir a visualização da agenda e filtragem por letras ao digitar o nome.
- RF3 - O sistema não apaga dados, mas realiza Inativação Lógica na agenda

Requisitos Não-Funcionais (RNFs):

- RNF1 - A persistência ocorrerá de forma transiente em um arquivo local texto (.txt) gerido de forma central.
- RNF2 - O sistema operará usando Java Swing e será encapsulado pelo Padrão MVC e result responses.

4.1 Casos de Uso significantes para a arquitetura

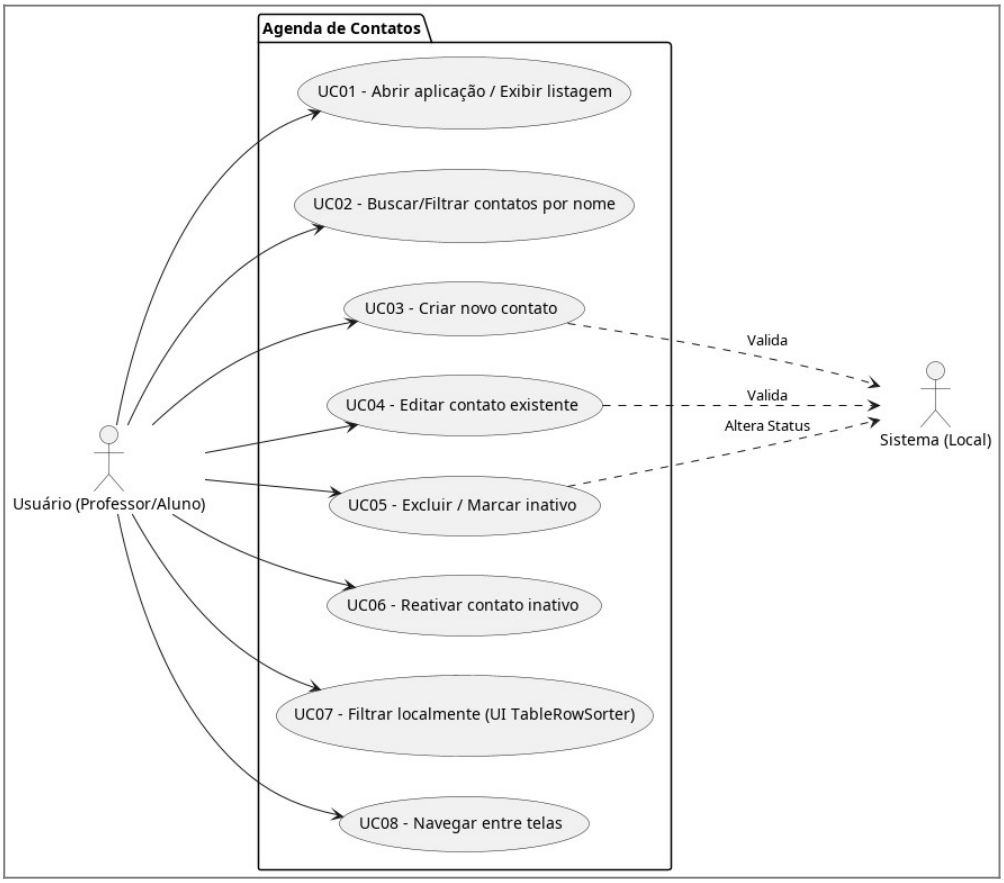

Figura 2 – Diagrama com os casos de uso significativos e atores

Matriz de Rastreabilidade (RF -> UC):

- RF1 Mapeado para -> UC03 (Criar) e UC04 (Editar)
- RF2 Mapeado para -> UC01 (Listar) e UC02 (Buscar)
- RF3 Mapeado para -> UC05 (Inativar) e UC06 (Reativar)

5. VISÃO LÓGICA

5.1 Panorama geral

A aplicação adota separação 4-Tier: Camada de Apresentação (Forms passivos), Camada de Controladoras (Guiada por Responses tipados), Camada de Serviços/Domínios (Regras e Validações isoladas) e Camada de Repositório (Lidando com a persistência de arquivo de forma exclusiva).

5.2 Camadas e responsabilidades

5.2.1 *Apresentação (UI)*

O Form_Listagem e Form_Cadastro são telas "passivas". Elas não possuem lógica se quer, baseando-se em envelopamentos (Response) despachados pelo Controller.

5.2.2 *Controllers*

O ContatoController centraliza as assinaturas (IContatoCadastroController, IContatoListaController), abstrai Exceções em Respostas Visuais limpas (Response.erro).

5.2.3 *Serviços (Domain/Service)*

O ContatoService blinda a inteligência. Nenhuma classe salva sem a autorização do ContatoValidation (que barra nomes ou emails errados lançando exceptions puras).

5.2.4 *Persistência (Repositories)*

Todo I/O do agenda.txt pertence unicamente à classe ContatoRepository. (Arquitetura formalizada na ADR-0002).

5.3 Organização em pacotes / subsistemas

- com.br.senai.ads3.agenda_fatesg.pages (UI)
- com.br.senai.ads3.agenda_fatesg.controllers (MVC e Respostas)
- com.br.senai.ads3.agenda_fatesg.services (Regras de Negócio)
- com.br.senai.ads3.agenda_fatesg.validations (Políticas de Validação de Input)
- com.br.senai.ads3.agenda_fatesg.repositories (Acesso a dados java.nio.file)
- com.br.senai.ads3.agenda_fatesg.domains (Entidade Contato)
- com.br.senai.ads3.agenda_fatesg.exceptions (Erros controlados: ExceptionValidationCampo e ExceptionValidationRegra)
- com.br.senai.ads3.agenda_fatesg.dtos (Wrappers genéricos Response)

5.4 Coesão e acoplamento

Alta coesão com Princípio da Responsabilidade Única (SRP): Cada classe cuida apenas de sua atribuição técnica.

Baixo acoplamento: O Controller envia DTOs e não lê mais arquivos (mudança implementada pela ADR-0001 e ADR-0002). A injeção nas camadas é feita 100% via Interfaces (Ex: IContatoService e IContatoRepository).

5.5 Classes mais importantes

5.5.1 *Form_Listagem / Form_Cadastro*

Telas UI.

5.5.2 *IContatoCadastroController e IContatoListaController (interface)*

Define a portabilidade dos contratos para os Form views. Métodos: criar(Contato), listarTodos(), excluir(String).

5.5.3 *ContatoController*

Implementa os contratos e repassa os fluxos interceptando exceções de validação.

5.5.4 *ContatoService e ContatoValidation*

Miolo blindado da aplicação. O Service só persiste se o Validation garantir que não há erro de Campo nem Quebra de Regra.

5.5.5 *ContatoRepository*

Manipulador implacável de Leitura, Gravação e deleção lógica (flag inativo) de contatos persistidos no disco.

5.6 Realização dos casos de uso

5.6.1 UC03 Criar Contato:

A UI preenche o Contato DTO → ContatoController.criar(contato) → O ContatoService.inserir(contato) delega a aprovação para ContatoValidation → Se ocorrer falha ou duplicidade o processo vomita um ExceptionValidationRegra que é encapsulado pelo Controller no objeto Response. Se der tudo OK, o ContatoRepository injeta fidedignamente no CSV via java.nio e a Listagem é atualizada.

5.6.2 UC01 Listar Contatos:

A UI dispara a listagem → ContatoController.listarTodosAtivos() é exigida → ContatoRepository lista e despreza os marcados como soft-deleted no txt.ontroller em bases maiores.

5.6.3 UC05 Excluir (marcar inativo)

Interface exige exclusão de "Joao" → A ponte cai até ContatoRepository.desativar(...) alterando apenas o flag inativo (ADR-0000 - Exclusão Lógica) sem corromper I/O contínuo.

5.7 Concorrência / performance

Para arquivos e listas de Contato muito extensas, o JTable pode sofrer travamentos de frame (UI freeze). O EDT (Exception Dispatch Thread) do Swing é livrado delegando grandes operações para SwingWorker.

- A persistência no TXT roda através de blocos e appends ágeis em java.nio limitando a sobrecarga de memória.

5.8 Visão Geral – pacotes e camadas

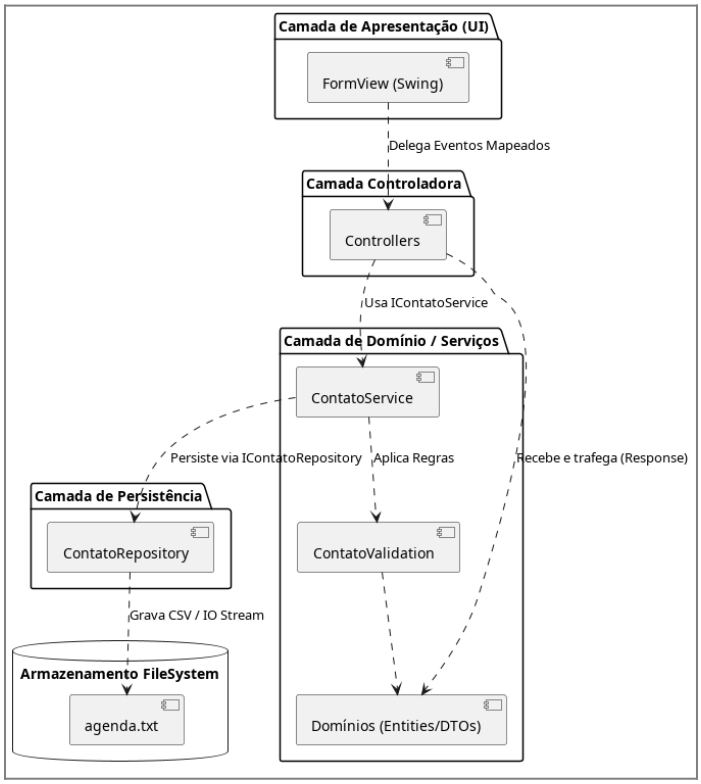

Figura 2 – Diagrama de Camadas da Aplicação

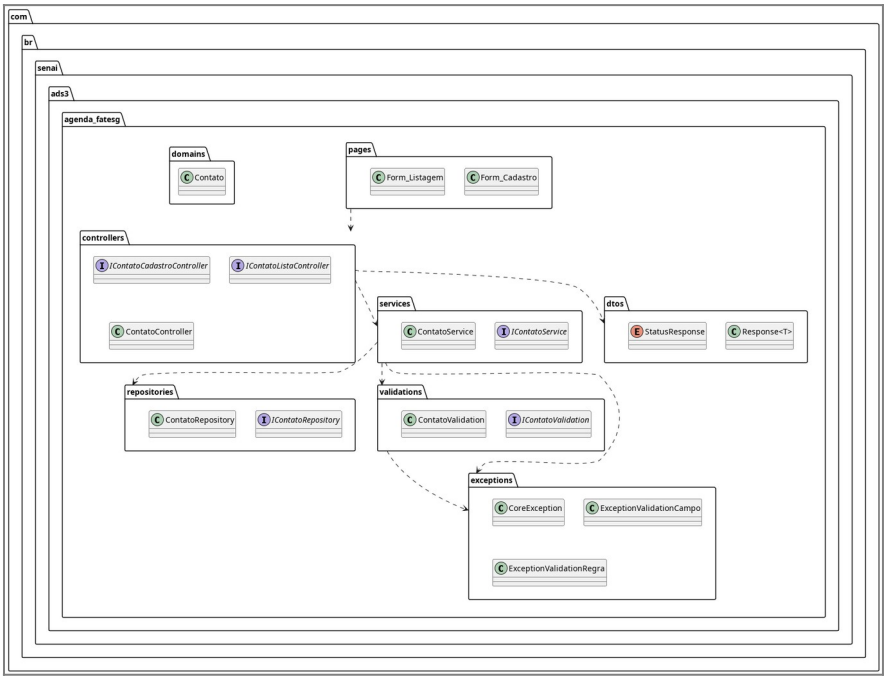

Figura 3 – Diagrama de Pacotes da Aplicação

6. VISÃO DE IMPLEMENTAÇÃO

6.1 Caso de Uso [UC03 - Criar Novo Contato]

6.1.1 Diagrama de Classes

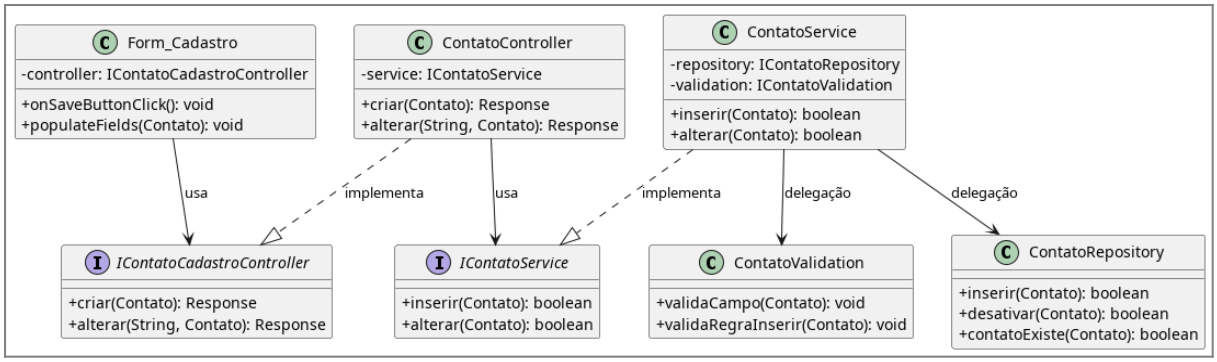

Figura 20 – Exemplo de Diagrama de Classes

6.1.2 Diagrama de Sequência

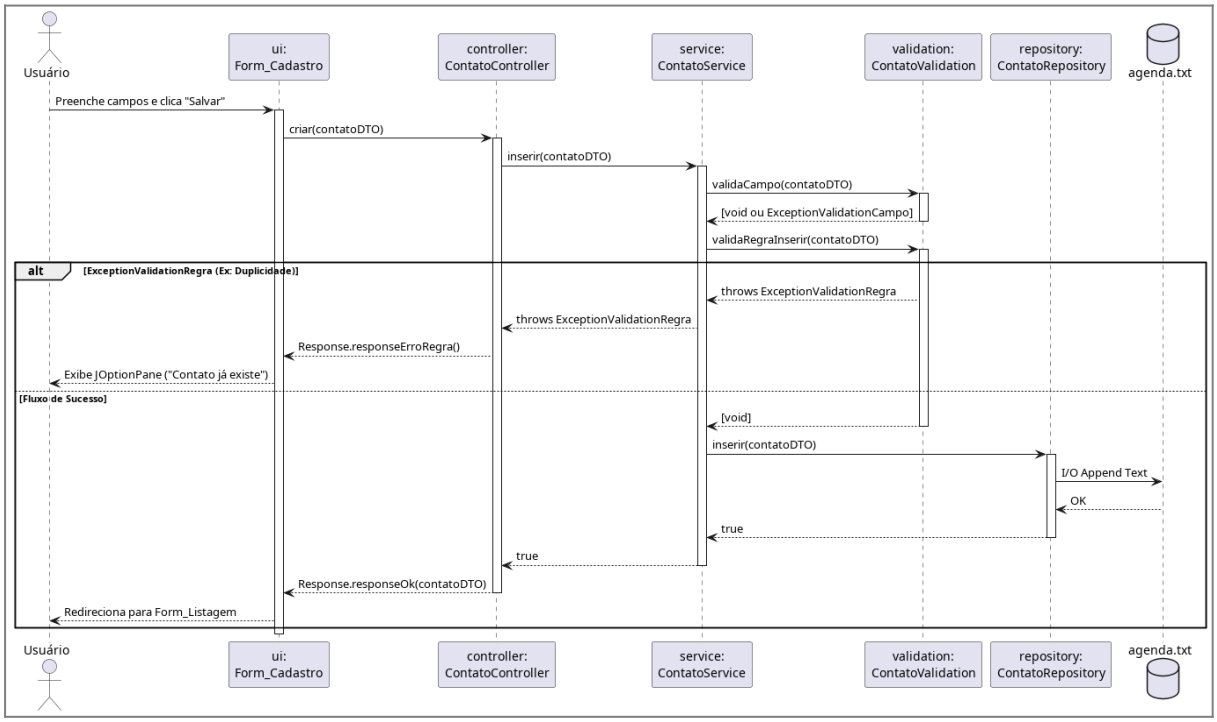

Figura 20 – Diagrama de Sequência

7. VISÃO DE IMPLANTAÇÃO

Descrever os modos físicos, as configurações e os artefatos que serão implantados.

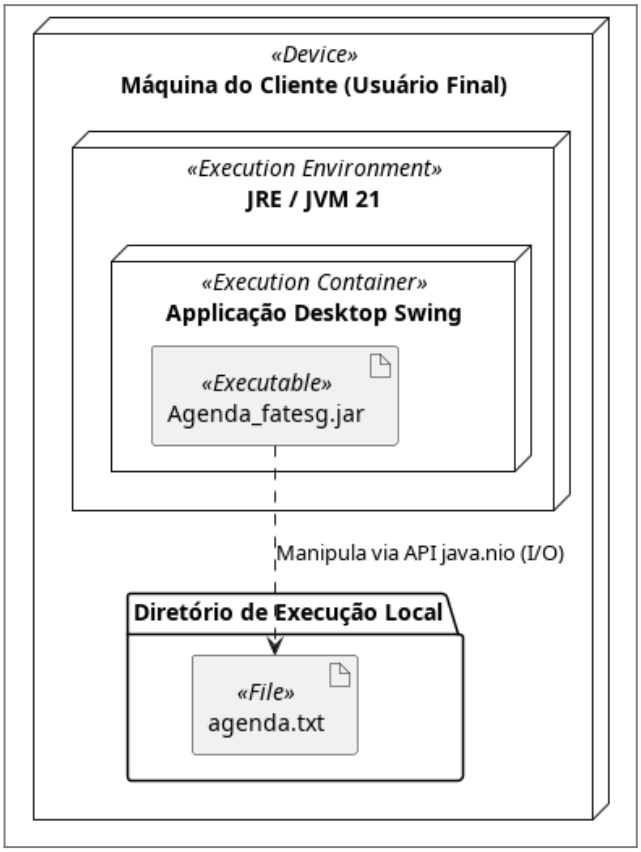

Figura 20 – Exemplo de Diagrama de Implantação Java

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume (estimativas e contexto)

- Número estimado de usuários (por instalação/cliente): 1 (aplicação desktop single-user).
- Número estimado de usuários simultâneos (em sala/ambiente): até 30 instalações simultâneas (cada aluno em sua máquina) — não há modo servidor central.
- Número estimado de acessos diários (por usuário): 10–50 operações (abrir app, listar, criar/editar contatos).
- Número estimado de acessos por período (picos em sala): durante aula pode haver 30 usuários realizando 1–3 operações por minuto.
- Tempo de sessão de um usuário: tipicamente 15–120 minutos (dependendo da atividade).

Observações:

- Como a persistência é isolada localmente (arquivo agenda.txt persistido via ContatoRepository), não há concorrência de acesso a nível de rede.
- Para futura evolução com backend compartilhado, estes números servirão como base para o dimensionamento do throughput da API REST.

8.2 Performance (metas e requisitos)

- Tempo máximo aceitável para operações comuns (meta UX):
- Leitura/listagem local (listarTodosAtivos): 200–500 ms para arquivos de até 5.000 registros.
- Inserção simples (create com FileWriter/append): ≤ 200 ms.
- Atualização (update lógico isolando ID via inativo=true): ≤ 500–800 ms.
- Filtragem local (UI RowFilter implementada na Swing): resposta interativa (<100 ms) via cache do TableModel.

Requisitos adicionais:

- UI não deve travar: operações de leitura síncrona espessa na inicialização não devem exceder 1.5s (ideal < 1s). O uso do EDT deve ser aliviado usando threads para listagens densas.

Critérios de aceitação:

- Testes de carga local com arquivo sintético de 5.000 registros mantêm listAll ≤ 500 ms e update ≤ 800 ms em máquina de desenvolvimento padrão (ex.: CPU i5, SSD, 8GB RAM).
- UI permanece responsiva (não bloqueada) durante operações de I/O em testes automatizados/locais.

8.3 Medição e monitoramento (como validar)

Estratégia de testes:

- Estratégia de testes: Massas sintéticas de N registros manipulando o ExceptionValidationRegra.
- Ferramentas: Logs temporários com timestamps de System.currentTimeMillis() em pontos críticos do Service e Repository ao simular I/O.

8.4 Estratégias de otimização (curto e médio prazo)

Curto prazo (Alcançado nesta iteração):

- Evitou-se reescrever o arquivo sempre que possível (Exclusão é Soft Delete [ADR-0000] e Inserções contínuas usam buffers append atômicos).
- Médio prazo (Futuro):
- Uma vez que o IContatoRepository foi blindado (ADR-0002), a migração em massa para SQLite/H2 poderá ser feita no futuro apenas trocando a classe de injeção, reduzindo absurdamente custos em milissegundos nas querys.

8.5 Riscos de performance e mitigação

- Arquivos superdensos (≥100k): Leitura síncrona lenta via java.nio. Mitigação → Paginação em memória (Cache).
- I/O no EDT bloqueando UI (UI freeze): Mitigação → Isolamento nas camadas profundas e requisição via SwingWorkers guiados a partir da camada Controller.
- Corrupções no arquivo: Mitigação → Operações atômicas de manipulação textual que já estão confinadas no ContatoRepository.

8.6 Efetividade na Execução

- A testabilidade garantida (seção Qualidade) possibilita rodar execuções de JMH sobre a classe Validation e o injetável Service, gerando os relatórios estáticos de I/O em máquina de desenvolvimento padrão.

8.7 Valores base para relatórios arquiteturais

- Nº estimado usuários (por instalação): 1
- Nº estimado acessos diários por usuário: 10–50
- Tempo máximo tolerado de carregamento listAll (em 5k regs): 500 ms
- Custo em tempo para Update lógico: 800 ms

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	Capacidade de suportar aumento no volume de dados e número de operações por instalação; atualmente aplicação é single-user local, mas deve permitir fácil migração para backend compartilhado.	Projetar a camada de Repositório (ContatoRepository) com I/O escalável (append de buffer para inserts), minimizando reescritas constantes do arquivo. A adoção da interface IContatoRepository permite migração futura imediata para bancos SQLite ou Cloud sem qualquer quebra do sistema.
Confiabilidade, Disponibilidade	Aplicação deve operar sem perda de dados e recuperar-se de falhas locais; disponibilidade depende da JVM/host local	Blindar a comunicação da interface com o "Unified Response Pattern" (ADR-0004) atuando como Result Object. Todas e quaisquer falhas despencam na camada correta (CoreException) impedindo travas e perdas fatais no uso do Form através de tratamento responsivo centralizado.
Portabilidade	A aplicação deve rodar em diferentes sistemas operacionais que suportem JVM 21 (Windows, Linux, macOS)	Manter o ecossistema puramente em Java 21, utilizando apenas Swing e java.nio (FileSystem) empacotado em executável JAR, preservando a interoperabilidade entre OS sem dependências de frameworks engessados.
Segurança	Proteção básica dos dados pessoais armazenados localmente (nome, e-mail, telefone) e contra entradas maliciosas; aplicação local sem autenticação central	A validação e sanitização não pertencem mais ao Controller, sendo impostas duramente por classes cegas de negócio (ContatoValidation e ContatoService) via Exceptions puras de quebra de regras (ADR-0003), barrando qualquer ameaça antes que alcance o fluxo em disco.
Manuenibilidade	Facilidade de alterar e estender o sistema (refactor, extração de Repository/Service)	Enforçar arquitetura em múltiplas camadas finas (UI, Control, Service, Repo). Controllers atuando puramente como passadores de mensagens 100% isentos de negócio. Todo avanço arquitetural rastreado e justificado via Architectural Decision Records (ADRs).
Testabilidade	Facilidade de escrever e executar testes automatizados para lógica de negócio	A ponte de Injeção de Dependências no IContatoRepository permite o isolamento de testes sem precisar gravar em disco real (Mocks). E como a classe ContatoValidation atua totalmente separada das interfaces JFrames ou do Swing, as lógicas de negócio e regras podem ser exaustivamente testadas como Unitárias em modo Headless.